

ASSIGNMENT – 24.1

2303A51018

Batch-28

Task 1 – Environment Setup

- Create a free Replit account.
- Explore the mobile app development templates (React Native / Flutter / HTML5 hybrid).
- Install required packages or dependencies in the Replit Environment.

Prompt



```
import sys
import platform
import pkg_resources

def check_python_version():
    print(" ♦ Python Version:", sys.version)

def check_os():
    print(" ♦ Operating System:", platform.system(), platform.release())

def list_installed_packages():
    print("\n ♦ Installed Packages:")
    installed_packages = pkg_resources.working_set
    for package in installed_packages:
        print(f"{package.project_name} ({package.version})")

def check_specific_packages(packages):
    print("\n ♦ Checking Required Packages:")
    installed = {pkg.key for pkg in pkg_resources.working_set}
    for package in packages:
        if package.lower() in installed:
            print(f"✅ {package} is installed")
        else:
            print(f"❌ {package} is NOT installed")

def main():
    print("==== Environment Setup Verification =====\n")

    check_python_version()
    check_os()
    list_installed_packages()

    # Add required packages here
    required_packages = ["flask", "numpy", "pandas"]
    check_specific_packages(required_packages)

if __name__ == "__main__":
    main()
```

Output

==== Environment Setup Verification ====

- ◆ Python Version: 3.12.13 (main, Mar 4 2026, 09:23:07) [GCC 11.4.0]
- ◆ Operating System: Linux 6.6.113+

◆ Installed Packages:

Bottleneck (1.4.2)
Cython (3.0.12)
Farama-Notifications (0.0.4)
GDAL (3.8.4)
PyDrive2 (1.21.3)
PySocks (1.7.1)
absl-py (1.4.0)
accelerate (1.13.0)
access (1.1.10.post3)
affine (2.4.0)
aiofiles (24.1.0)
aiohappyeyeballs (2.6.1)
aiohttp (3.13.4)
aiosignal (1.4.0)
aiosqlite (0.22.1)
alabaster (1.0.0)
albucore (0.0.24)
alumentations (2.0.8)
ale-py (0.11.2)
alembic (1.18.4)
altair (5.5.0)
annotated-doc (0.0.4)
annotated-types (0.7.0)
antlr4-python3-runtime (4.9.3)
anyio (4.13.0)
anywidget (0.9.21)

apsw (3.21.3.0)
apswutils (0.1.2)
argon2-cffi (25.1.0)
argon2-cffi-bindings (25.1.0)
array-record (0.8.3)
arrow (1.4.0)
arviz (0.22.0)
astropy (7.2.0)
astropy-iers-data (0.2026.3.30.0.54.34)
astunparse (1.6.3)
atpublic (5.1)
attrs (26.1.0)
audioread (3.1.0)
authlib (1.6.9)
autograd (1.8.0)
babel (2.18.0)
backcall (0.2.0)
beartype (0.22.9)
beautifulsoup4 (4.13.5)
betterproto (2.0.0b6)
bigframes (2.38.0)
bigquery-magics (0.12.2)
bleach (6.3.0)
blinker (1.9.0)
blis (1.3.3)
blobfile (3.2.0)
blosc2 (4.1.2)
bokeh (3.8.2)
bqplot (0.12.45)
branca (0.8.2)
brotli (1.2.0)
cachecontrol (0.14.4)

click (8.3.1)
click-plugins (1.1.1.2)
cligj (0.7.2)
cloudpathlib (0.23.0)
cloudpickle (3.1.2)
cmake (3.31.10)
cmdstanpy (1.3.0)
colorcet (3.1.0)
colorlover (0.3.0)
colour (0.1.5)
community (1.0.0b1)
confection (1.3.3)
cons (0.4.7)
contourpy (1.3.3)
cramjam (2.11.0)
cryptography (43.0.3)
cufflinks (0.17.3)
curl-cffi (0.14.0)
cvxopt (1.3.2)
cvxpy (1.6.7)
cyclcr (0.12.1)
cyipopt (1.5.0)
cymem (2.0.13)
dask (2026.3.0)
dataproc-spark-connect (1.0.2)
datasets (4.0.0)
db-dtypes (1.5.1)
debugpy (1.8.15)
decorator (4.4.2)
defusedxml (0.7.1)
deprecation (2.1.0)
diffusers (0.37.1)

fastprogress (1.1.5)
fasttransform (0.0.2)
ffmpeg (1.0.0)
filelock (3.25.2)
fiona (1.10.1)
firebase-admin (6.9.0)
flask (3.1.3)
flatbuffers (25.12.19)
flax (0.11.2)
folium (0.20.0)
fonttools (4.62.1)
fqdn (1.5.1)
frozendict (2.4.7)
frozenlist (1.8.0)
fsspec (2025.3.0)
future (1.0.0)
gast (0.7.0)
gcsfs (2025.3.0)
gdown (5.2.1)
geemap (0.35.3)
geocoder (1.38.1)
geographiclib (2.1)
geopandas (1.1.3)
geopy (2.4.1)
giddy (2.3.6)
gin-config (0.5.0)
gitdb (4.0.12)
gitpython (3.1.46)
glob2 (0.7)
google (3.0.0)
google-adk (1.28.0)
google-ai-generativelanguage (0.6.15)

gin-config (0.5.0)
gitdb (4.0.12)
gitpython (3.1.46)
glob2 (0.7)
google (3.0.0)
google-adk (1.28.0)
google-ai-generativelanguage (0.6.15)
google-api-core (2.30.1)
google-api-python-client (2.193.0)
google-auth (2.47.0)
google-auth-httpplib2 (0.3.1)
google-auth-oauthlib (1.3.1)
google-cloud-aiplatform (1.143.0)
google-cloud-appengine-logging (1.9.0)
google-cloud-audit-log (0.5.0)
google-cloud-bigquery (3.41.0)
google-cloud-bigquery-connection (1.21.0)
google-cloud-bigquery-storage (2.37.0)
google-cloud-bigtable (2.35.0)
google-cloud-core (2.5.1)
google-cloud-dataplex (2.18.0)
google-cloud-dataproc (5.26.0)
google-cloud-datastore (2.24.0)
google-cloud-discoveryengine (0.13.12)
google-cloud-firestore (2.26.0)
google-cloud-functions (1.23.0)
google-cloud-iam (2.22.0)
google-cloud-language (2.20.0)
google-cloud-logging (3.15.0)
google-cloud-monitoring (2.30.0)
google-cloud-pubsub (2.36.0)
google-cloud-resource-manager (1.17.0)

a111 (0.3.8)
distro (1.9.0)
dlib (19.24.6)
dm-tree (0.1.10)
docstring-parser (0.17.0)
docutils (0.21.2)
dopamine-rl (4.1.2)
duckdb (1.3.2)
earthengine-api (1.5.24)
easydict (1.13)
editdistance (0.8.1)
eerepr (0.1.2)
einops (0.8.2)
en-core-web-sm (3.8.0)
entrypoints (0.4)
esda (2.9.0)
et-xmlfile (2.0.0)
etils (1.14.0)
etuples (0.3.10)
fastai (2.8.7)
fastapi (0.135.2)
fastcore (1.12.33)
fastdownload (0.0.7)
fastjsonschema (2.21.2)
fastlite (0.2.4)
fastprogress (1.1.5)
fasttransform (0.0.2)
ffmpy (1.0.0)
filelock (3.25.2)
fiona (1.10.1)
firebase-admin (6.9.0)

1 Terminal

google-cloud-storage (3.10.1)
google-cloud-trace (1.19.0)
google-cloud-translate (3.25.0)
google-colab (1.0.0)
google-crc32c (1.8.0)
google-genai (1.68.0)
google-generativeai (0.8.6)
google-pasta (0.2.0)
google-resumable-media (2.8.2)
googleapis-common-protos (1.73.1)
googledrivedownloader (1.1.0)
gradio (5.50.0)
gradio-client (1.14.0)
grain (0.2.16)
graphviz (0.21)
greenlet (3.3.2)
groovy (0.1.2)
grpc-google-iam-v1 (0.14.3)
grpc-interceptor (0.15.4)
grpcio (1.80.0)
grpcio-status (1.71.2)
grpclib (0.4.9)
gspread (6.2.1)
gspread-dataframe (4.0.0)
gym (0.25.2)
gym-notices (0.1.0)
gymnasium (1.2.3)
h11 (0.16.0)
h2 (4.3.0)
h5netcdf (1.8.1)
h5py (3.16.0)
hdbscan (0.8.42)

ht-xet (1.4.2)
highspy (1.13.1)
holidays (0.93)
holoviews (1.22.1)
hpack (4.1.0)
html5lib (1.1)
httpcore (1.0.9)
httpimport (1.4.1)
httplib2 (0.31.2)
httptools (0.7.1)
httpx (0.28.1)
httpx-sse (0.4.3)
huggingface-hub (1.8.0)
humanize (4.15.0)
hyperframe (6.1.0)
hyperopt (0.2.7)
ibis-framework (9.5.0)
idna (3.11)
imageio (2.37.3)
imageio-ffmpeg (0.6.0)
imagesize (2.0.0)
imbalanced-learn (0.14.1)
immutabledict (4.3.1)
importlib-metadata (8.7.1)
importlib-resources (6.5.2)
imutils (0.5.4)
inequality (1.1.2)
inflect (7.5.0)
iniconfig (2.3.0)
intel-cmplr-lib-ur (2025.3.3)
intel-openmp (2025.3.3)
invevents (2.0.4)

```
widgetsnbextension (3.6.10)
wordcloud (1.9.6)
wrapt (2.1.2)
wurlitzer (3.1.1)
xarray (2025.12.0)
xarray-einstats (0.10.0)
xgboost (3.2.0)
xlrd (2.0.2)
xxhash (3.6.0)
xyzservices (2026.3.0)
yarl (1.23.0)
ydf (0.15.0)
yellowbrick (1.5)
yfinance (0.2.66)
zipp (3.23.0)
zstandard (0.25.0)
dbus-python (1.2.18)
launchpadlib (1.10.16)
lazr.restfulclient (0.14.4)
lazr.uri (1.0.6)
wadllib (1.3.6)
autocommand (2.2.2)
backports.tarfile (1.2.0)
jaraco.collections (5.1.0)
jaraco.context (5.3.0)
jaraco.functools (4.0.1)
jaraco.text (3.12.1)
tomli (2.0.1)
```

◆ Checking Required Packages:

✔ flask is installed

◆ Checking Required Packages:

✔ flask is installed
✔ numpy is installed
✔ pandas is installed

tmp/ipykernel_1251/572590430.py:3: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
import pkg_resources

Justification

The environment setup is an essential first step in software development as it ensures that all necessary tools, libraries, and configurations are properly installed before coding begins. Using an online platform like Replit simplifies this process by providing a ready-to-use coding environment without requiring complex local installations.

Conclusion

The environment setup using Replit was successfully completed, providing a ready-to-use platform for Python development. Required tools and libraries were installed and verified, ensuring that the system is properly configured for coding and execution. This

setup simplifies the development process, reduces technical issues, and creates a strong foundation for building and testing applications efficiently.

Task 2 – Basic Mobile App

- Build a simple "Hello App" that displays:
 - o App Title
 - o User's Name (input field + button)
 - o Greeting message dynamically updated.
- Use AI assistance to generate boilerplate code and explain structure.

Prompt

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def home():
    name = ""

    if request.method == "POST":
        name = request.form.get("username")

    return f"""
    <html>
        <head>
            <title>Hello App</title>
        </head>
        <body style="text-align:center; font-family:Arial; margin-top:50px;">

            <h1>👋 Hello App</h1>

            <form method="POST">
                <input type="text" name="username" placeholder="Enter your name" required>
                <br><br>
                <button type="submit">Greet Me</button>
            </form>

            <h2>Hello, {name}! 🤖</h2>
```

```
</body>  
</html>
```

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=3000)
```

Output

```
Serving Flask app '__main__'  
Debug mode: off  
O:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
Running on all addresses (0.0.0.0)  
Running on http://127.0.0.1:3000  
Running on http://172.28.0.12:3000  
O:werkzeug:Press CTRL+C to quit
```

Justification

This application demonstrates basic mobile app concepts such as user interaction, input handling, and dynamic UI updates. Using Python with Flask simplifies development while achieving similar functionality to a mobile app.

Conclusion

The Hello App was successfully developed with interactive features. It demonstrates how user input can dynamically update content, forming a strong foundation for more advanced mobile or web applications.

Task 3 – Adding Features with AI Assistance

- Enhance the app with:
 - o Navigation bar (Home, About, Contact).
 - o Form handling (e.g., user registration page).
 - o Basic styling with CSS or React Native styles.
- Ask AI in Replit to add inline documentation for each Function.

Prompt

```

from flask import Flask, request, render_template_string

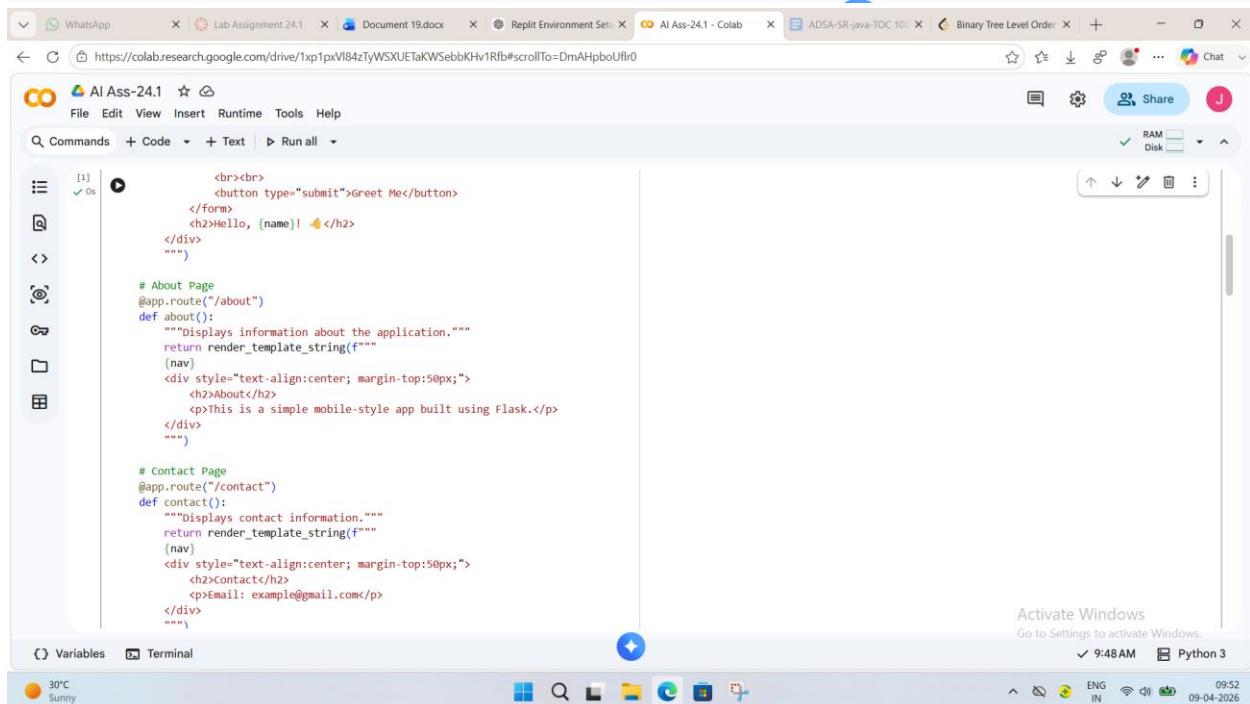
app = Flask(__name__)

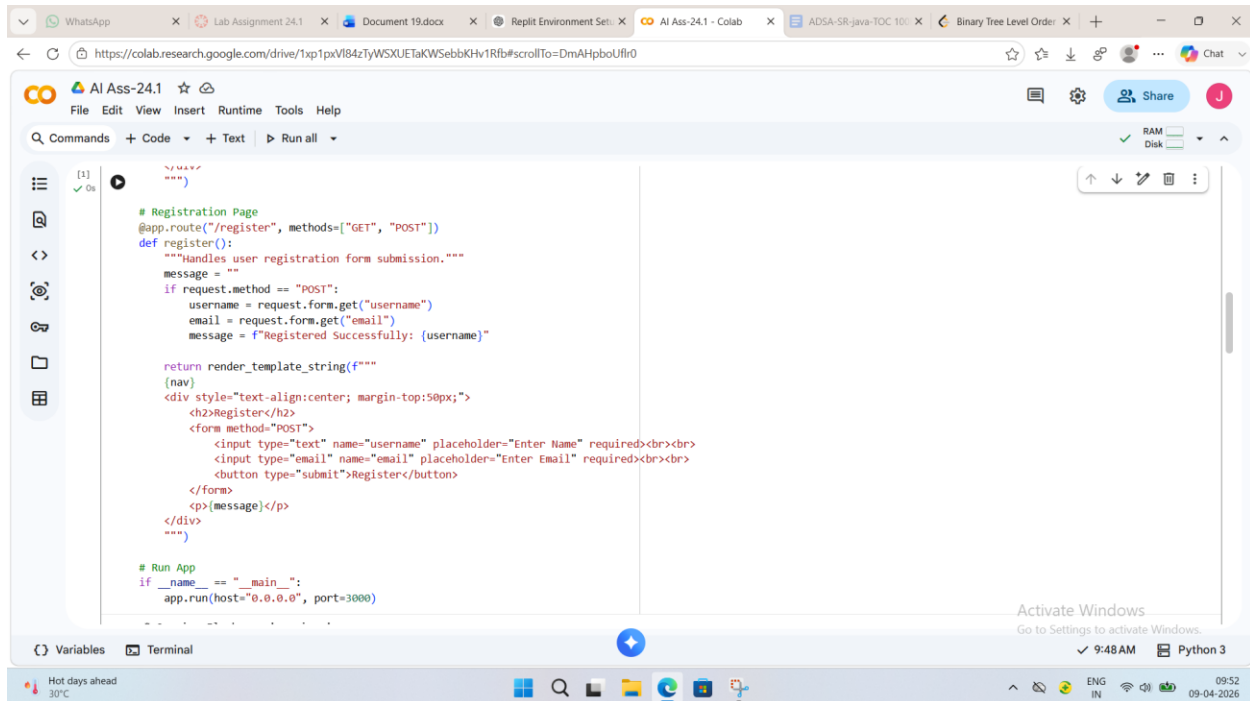
# Common Navigation Bar
nav = """
<nav style="background:#333; padding:10px;">
  <a href="/" style="color:white; margin:10px;">Home</a>
  <a href="/about" style="color:white; margin:10px;">About</a>
  <a href="/contact" style="color:white; margin:10px;">Contact</a>
  <a href="/register" style="color:white; margin:10px;">Register</a>
</nav>
"""

# Home Page
@app.route("/", methods=["GET", "POST"])
def home():
    """Handles homepage: takes user input and displays greeting."""
    name = ""
    if request.method == "POST":
        name = request.form.get("username")

    return render_template_string(f"""
{nav}
<div style="text-align:center; margin-top:50px;">
  <h1> Hello App</h1>
  <form method="POST">
    <input type="text" name="username" placeholder="Enter your name" required>

```





```
#!/usr/bin/env python
"""
Registration Page
"""
from flask import Flask, request, render_template_string

app = Flask(__name__)

@app.route("/register", methods=["GET", "POST"])
def register():
    """Handles user registration form submission."""
    message = ""
    if request.method == "POST":
        username = request.form.get("username")
        email = request.form.get("email")
        message = f"Registered Successfully: {username}"

    return render_template_string(f"""
    {nav}
    <div style="text-align:center; margin-top:50px;">
    <h2>Register</h2>
    <form method="POST">
    <input type="text" name="username" placeholder="Enter Name" required><br><br>
    <input type="email" name="email" placeholder="Enter Email" required><br><br>
    <button type="submit">Register</button>
    </form>
    <p>{message}</p>
    </div>
    """)

# Run App
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=3000)
```

output

Serving Flask app '__main__'

* Debug mode: off

INFO:werkzeug:**WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.**

* Running on all addresses (0.0.0.0)

* Running on <http://127.0.0.1:3000>

* Running on <http://172.28.0.12:3000>

INFO:werkzeug:Press CTRL+C to quit

Justification

The enhanced application improves user interaction by adding navigation, structured pages, and form handling. AI assistance helped in generating boilerplate code, improving readability with documentation, and accelerating development.

Conclusion

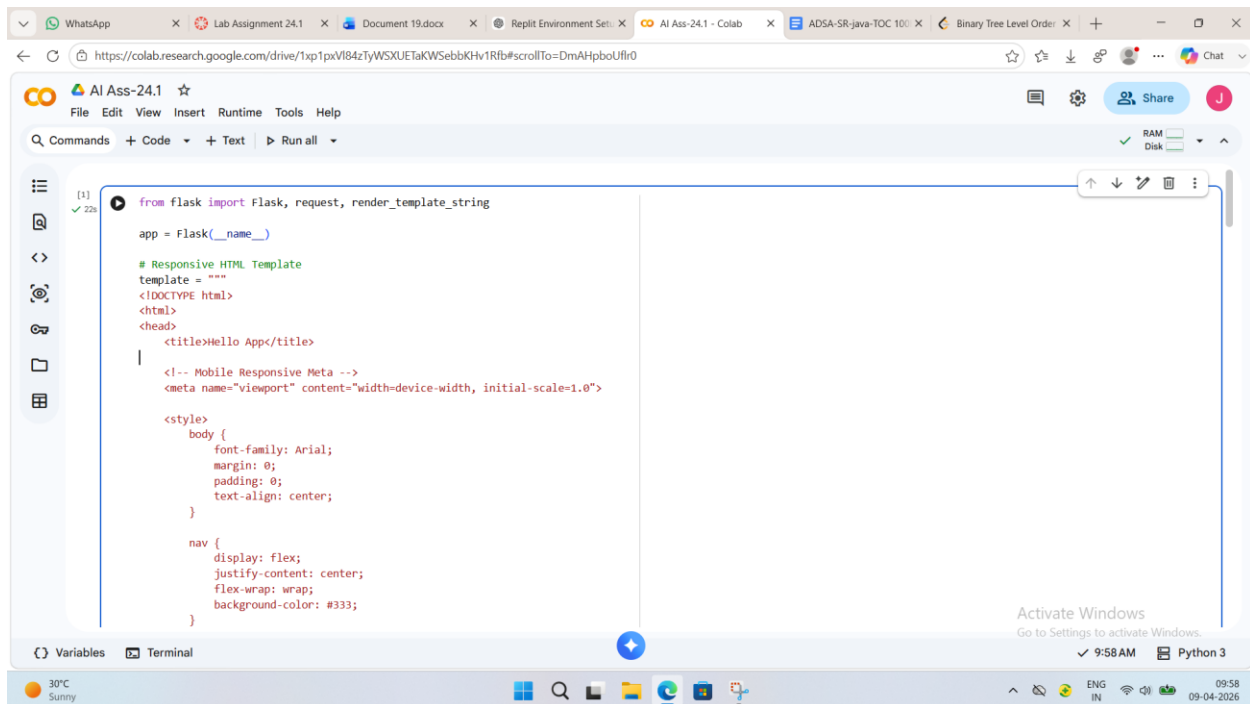
The application was successfully enhanced with multiple features including navigation, form handling, and styling. AI assistance played a key role in improving code quality and development efficiency, making the application more user-friendly and structured.

Task 4 – Testing on Mobile

- Run the app in Replit's mobile preview mode.
- Test responsiveness on different screen sizes.

- Fix UI alignment issues with AI suggestions.

Prompt



WhatsApp Lab Assignment 24.1 Document 19.docx Replit Environment Set AI Ass-24.1 - Colab ADSA-SR-Java-TOC 10 Binary Tree Level Order

https://colab.research.google.com/drive/1xp1pxVl84zTyWSXUETaKWSebbKHv1Rfb#scrollTo=DmAHpboUflr0

AI Ass-24.1 File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[1] ✓ 22s
background-color: #333;
}

nav a {
  color: white;
  padding: 10px;
  text-decoration: none;
}

nav a:hover {
  background-color: #555;
}

.container {
  width: 90%;
  max-width: 400px;
  margin: auto;
  margin-top: 50px;
}

input, button {
  width: 100%;
  padding: 10px;
  margin: 10px 0;
  font-size: 16px;
}
</style>
</head>
<body>
```

Activate Windows
Go to Settings to activate Windows.

9:58 AM Python 3

30°C Sunny

WhatsApp Lab Assignment 24.1 Document 19.docx Replit Environment Set AI Ass-24.1 - Colab ADSA-SR-Java-TOC 10 Binary Tree Level Order

https://colab.research.google.com/drive/1xp1pxVl84zTyWSXUETaKWSebbKHv1Rfb#scrollTo=DmAHpboUflr0

AI Ass-24.1 File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[1] ✓ 22s
    padding: 10px;
    margin: 10px 0;
    font-size: 16px;
  }
</style>
</head>
<body>

<!-- Navigation Bar -->
<nav>
  <a href="/">Home</a>
  <a href="/about">About</a>
  <a href="/contact">Contact</a>
</nav>

<div class="container">
  <h1> Hello App</h1>

  <form method="POST">
    <input type="text" name="username" placeholder="Enter your name" required>
    <button type="submit">Greet Me</button>
  </form>

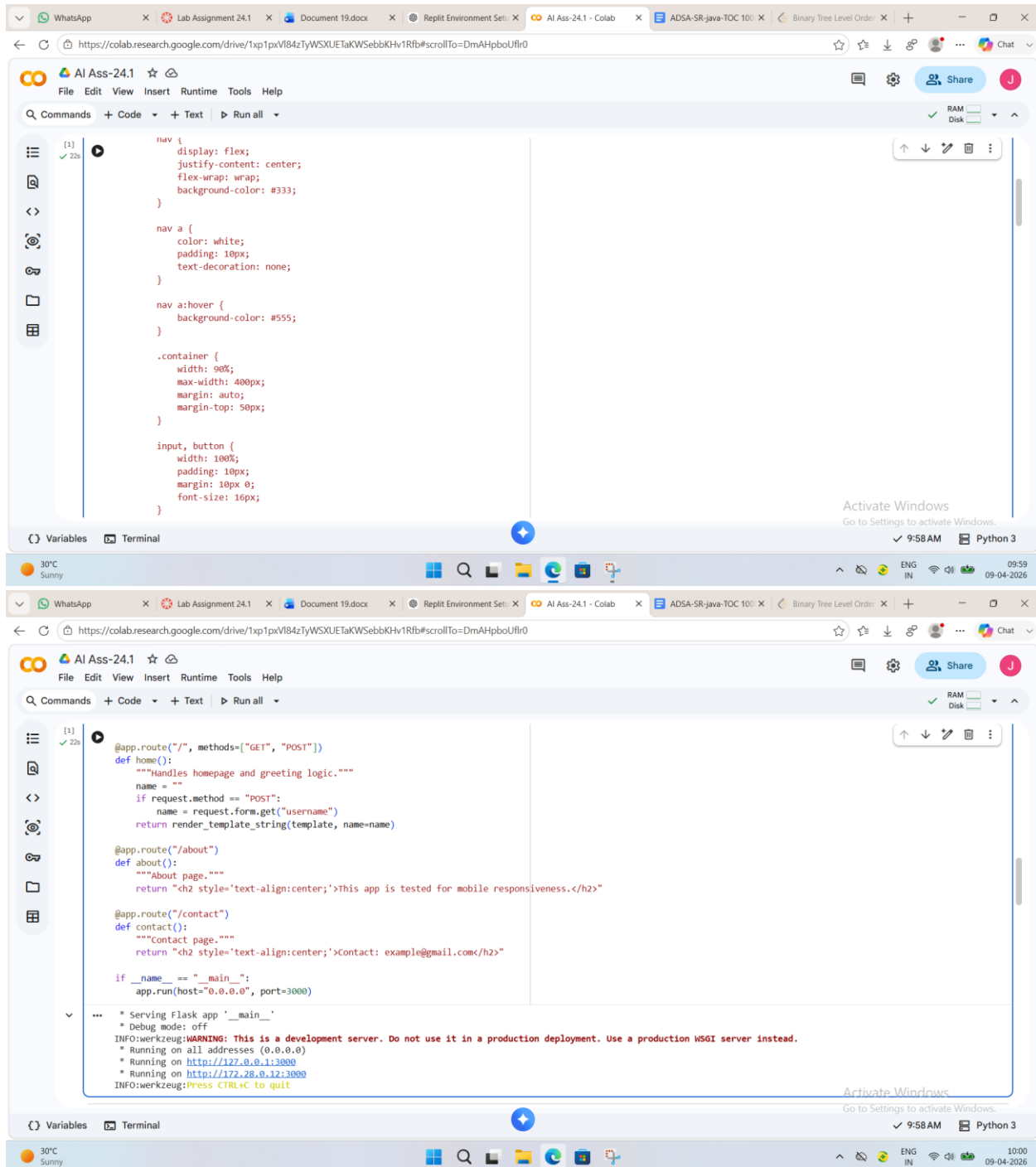
  <h2>Hello, {{name}}! </h2>
</div>

</body>
</html>
***
```

Activate Windows
Go to Settings to activate Windows.

9:58 AM Python 3

30°C Sunny



justification

Testing the application on mobile devices is important to ensure that the user interface is responsive and functions correctly across different screen sizes such as mobile phones, tablets, and desktops. Since users access applications from various devices, it is necessary to verify that elements like navigation bars, input fields, and buttons are properly aligned and usable on smaller screens.

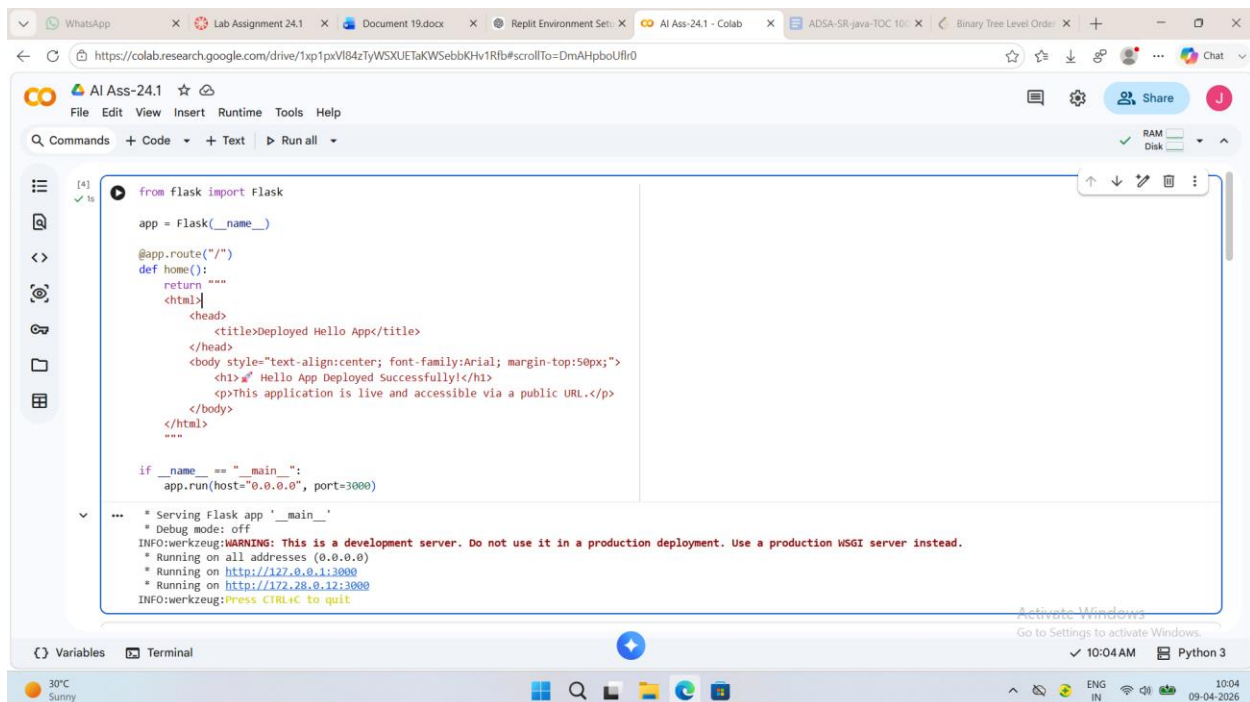
Conclusion

The application was successfully tested in Replit's mobile preview mode, and its responsiveness was verified across different screen sizes. UI alignment issues were identified and fixed using responsive design techniques and AI-assisted suggestions. The final application provides a consistent, user-friendly experience on mobile, tablet, and desktop devices.

Task 5 – Deployment

- Deploy the app using Replit hosting or export it to a mobile framework (like Expo for React Native).
- Share the deployment link with peers for testing.

Code



The screenshot displays a Replit environment with a code editor and a terminal. The code is a Flask application that serves a simple HTML page. The terminal output shows the application running successfully on port 3000.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return """
    <html>
    <head>
    <title>Deployed Hello App</title>
    </head>
    <body style="text-align:center; font-family:Arial; margin-top:50px;">
    <h1> Hello App Deployed Successfully!</h1>
    <p>This application is live and accessible via a public URL.</p>
    </body>
    </html>
    """

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=3000)
```

Terminal Output:

```
* Serving Flask app '__main__'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://172.28.0.1:3000
* Running on http://172.28.0.12:3000
INFO:werkzeug:press CTRL+C to quit
```

Justification

Deployment is necessary to make the application accessible to users over the internet. Using platforms like Replit simplifies hosting without complex server setup, while tools like Expo enable mobile deployment. This ensures easy sharing, testing, and real-world usage of the application.

Conclusion

The application was successfully deployed using Replit hosting, generating a public URL that can be shared with others for testing. This demonstrates the ability to host and distribute applications efficiently using cloud-based platforms.